

UCS503P - Software Engineering (Project)

Window

A Swipe-Based Local Marketplace for Android and the Web

Submitted by:

1024030167 Piyush Malik

1024030160 Paarth Singh

1024030550 Bhuvik Garg

Group: 3C16 BE Third Year, CSE

Submitted to:

Miss Paramveer Kaur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	5
1.1	Background and Context	5
1.1.1	Problem Statement	5
1.2	Project Objectives	6
2	Methodology and Design	7
2.1	System Architecture	7
2.1.1	High-Level Design	7
2.1.2	Technical Stack	7
2.2	Implementation Details	8
2.2.1	Module 1: Discovery Feed and Swipe Economy	8
2.2.2	Module 2: Likes, Matches and Chat	8
2.2.3	Module 3: Seller Onboarding and Product Creation	9
2.2.4	Module 4: Presentation and Platform	9
3	Results and Evaluation	11
3.1	Testing Methodology	11
3.2	Performance Metrics	12
3.3	Analysis	12
4	Conclusion	15
4.1	Summary of Work	15
4.2	Key Findings	15
4.3	Future Work and Recommendations	15
4.4	Final Remarks	16

Introduction

1.1 Background and Context

Small sellers in a city like Patiala, whether they stitch kurtas, sell juttis, paint, or furnish hostel rooms, rarely have a storefront that reaches beyond the street they are on. The marketplaces that do reach customers are built for warehouses: grids of thousands of listings, mandatory prices, carts, and a checkout that puts a company between the buyer and the person who made the thing. For a local seller that distance is the problem, not the solution. The buyer cannot ask whether a kurta comes in a larger size, the seller cannot say “come by the shop this evening”, and neither side knows whether the item on the screen still exists.

Window is a mobile-first marketplace that keeps the shop and the buyer within arm’s reach of each other. Products are shown one at a time as cards, like a dating app: a swipe to the left passes, a swipe to the right tells the seller you are interested, and the seller accepts the like to open a chat. Prices are a range or nothing at all; the conversation closes the sale. Everything is local: the feed only shows sellers within a radius of the buyer’s own area, and the radius widens only when there is not enough nearby.

The project was built by the three of us over the first half of the semester under the course’s constraints: ship a usable increment within two weeks, iterate weekly, and let continuous integration and delivery carry every release. The result is a website at `window-beige.vercel.app` and a signed Android application distributed through GitHub Releases, both built from one codebase.

1.1.1 Problem Statement

Buyers and local sellers lack a low-friction, trustworthy channel to discover each other and talk. Concretely:

- **Decision fatigue.** Catalogue grids present hundreds of choices at once; buyers scroll past most of them and remember none.
- **No direct line to the seller.** Questions about sizes, materials, delivery, or bargaining have nowhere to go.
- **Stale availability.** A listing says “in stock” long after the last piece is gone, and the buyer finds out only after ordering.

- **No locality.** Platforms show the same products to everyone, so a seller two streets away is buried under national inventory.

Window's hypothesis is that a fast, playful discovery loop (one card at a time), a direct chat opened by mutual interest, and live stock updates pushed to everyone who liked a product together turn browsing into conversations and conversations into local sales.

1.2 Project Objectives

1. **Objective 1 (discovery loop):** Deliver a working swipe feed where a right swipe reaches the seller and a seller's acceptance opens a chat, with a median swipe-to-match latency of at most 2 seconds measured on the client, within the first two-week iteration.
2. **Objective 2 (local, real-time marketplace):** Show only sellers within an adaptive radius (10, 25, then 50 km) of the buyer's chosen area, deliver chat messages in under 1 second, and propagate a seller's stock change to every interested buyer's device within seconds, by the mid-semester evaluation.
3. **Objective 3 (seller onboarding and tooling):** Let a seller open a shop from their phone in under five minutes (name, tags, map location, storefront photo, verified contact) and list a product with the camera, a crop step and a price range, with upload quotas and simulated purchases, by the mid-semester evaluation.
4. **Objective 4 (engineering process):** Keep every change behind an automated pipeline: linting, type checking, unit tests, security-rule tests and an end-to-end test on a local Firebase emulator on every push; automatic production deployment of the website; and a signed Android APK built and published for every tagged version.

These objectives are specific, measurable through instrumentation built into the application, achievable with managed services rather than research, relevant to the course's emphasis on time-to-value, and bounded to the first half of the semester.

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

Window is a single-page web application that is served in two ways: Vercel hosts it as the website, and Capacitor wraps the identical build into an Android application. There is no custom server. The client talks directly to Firebase Authentication and Cloud Firestore, and Firestore security rules decide who may read and write what. Images are uploaded from the device to Cloudinary through an unsigned preset; the database stores only their URLs. Maps come from OpenStreetMap tiles via Leaflet, with Nominatim for free geocoding.

This “no backend” shape was a deliberate trade-off. It let the three of us ship the first working version in one night and keeps the running cost at zero on free tiers. The cost is that fan-out logic that a server would normally perform (for example notifying every buyer who liked a product when its stock changes) is performed by the seller’s client in a single batched write, and the rules verify that the client is entitled to each document it touches. Firestore’s `getAfter()` is used so that a like and the match it creates can be validated together inside one atomic batch.

The application has two faces that share one account. The shopper face keeps the middle of the screen free for cards and puts its controls in the four corners: swipe and super-swipe balances top-left, a switch to the seller face top-right, liked products and chats bottom-left, and the profile bottom-right. The seller face has the same switch top-right, a plus button bottom-centre that opens the camera, shop management bottom-left and the seller profile bottom-right.

2.1.2 Technical Stack

- **Framework/Language:** React 19 with TypeScript, built by Vite; Tailwind CSS 4 for styling; `motion` for the card gestures and the flip, overlay and prompt animations; React Router for navigation.
- **Mobile:** Capacitor 8 wraps the web build into Android, with native plugins for geolocation, camera and Google Sign-In.
- **Database and authentication:** Cloud Firestore and Firebase Authentication (email/password and Google), with security rules kept in the repository and deployed by the pipeline.

- **Geospatial:** `geofire-common` geohashes on every product, Leaflet with OpenStreetMap tiles, and Nominatim for forward and reverse geocoding.
- **Images:** Cloudinary unsigned uploads after on-device compression and cropping (`react-easy-crop`).
- **Tools:** `pnpm`, `oxlint`, Vitest with Testing Library, the Firebase emulator suite with `@firebase/rules-unit-testing`, GitHub Actions, Vercel, and MkDocs for the documentation site.

2.2 Implementation Details

2.2.1 Module 1: Discovery Feed and Swipe Economy

Products are shown as a stack of three cards. The top card can be dragged, tapped to flip, or driven with the keyboard. Dragging left lights the left edge of the screen red and stamps the card NOPE; dragging right lights the right edge green and stamps it LIKE. The back of a card shows the price range, the full description, tags, the shop, and whether the product costs a swipe or a super swipe.

Every buyer receives five free swipes a day and two super swipes on the first login from the Android app. A left swipe is free and merely remembered so the card does not return. A right swipe costs one swipe and writes, in one batch, the swipe record, a *like* document in a pending state, and the buyer's new balance. A super swipe costs one super swipe and claims the product: the like starts accepted and the match document is created in the same batch, so the chat opens immediately. Products marked "super swipes only" refuse plain swipes, and the rules enforce this on the server side. When a buyer's balance reaches zero the counter grows and an overlay offers super swipes, bundles (simulated payment), or, on the website, the app with its two free super swipes.

The feed is local. Each product carries a geohash copied from its shop's pinned location. The client asks Firestore for the geohash ranges that cover the buyer's radius, filters by exact distance, availability and swipe history on the device, and ranks products whose tags match the buyer's chosen interests first and nearest first. If fewer than about twenty products are found the radius widens from 10 to 25 to 50 km, and a chip lets the buyer override it.

2.2.2 Module 2: Likes, Matches and Chat

A seller's home screen lists likes waiting for a decision. Accepting flips the like to accepted and creates the match document; declining closes it. A match holds denormalised copies of the product, the shop, both names and unread counters, and if the shop has configured an automatic message it is copied onto the match so the chat

opens with it. Shops can also buy a day of “auto-matching” (99.99 rupees, simulated) during which every right swipe matches instantly.

Chat is a messages subcollection under the match, read with real-time listeners. Sending a message is a batch that appends the message and updates the match’s last-message fields and the other party’s unread counter. Availability changes fan out from the seller’s client: the product is updated, every match on that product gets the new stock state, and every matched buyer receives a notification document, all in one batch that the rules verify match by match.

2.2.3 Module 3: Seller Onboarding and Product Creation

A seller sets up a shop in four steps shown on a progress rail: name, up to three tags and an optional website; location on a map with a GPS button, a place search and a draggable pin; a storefront photo; and the owner’s name, phone and email, each verified by a one-time code (the delivery of codes is simulated for the prototype and accepts 0000). Google Sign-In fills the owner’s name and verified email in one tap. A shopper who already has an account becomes a seller without creating a second account; the two faces share one identity.

The plus button opens a live camera preview (or the gallery). The photo is compressed on the device, cropped to a 3:4 frame, then described: tags from a curated vocabulary that deliberately excludes food, medical items, drugs and electronics; a name, for which a product code such as `WN-7F3K2Q` is generated on publish; a description limited to 150 words; a price range entered as two numbers that reveal a dual-thumb slider for fine-tuning; the shop, when the seller owns several; preset payment modes; and a “super swipes only” switch. Listings are metered: one upload token allows ten products, new sellers receive one, and bundles of 5, 10, 15 and 20 tokens are sold with the list price struck through.

2.2.4 Module 4: Presentation and Platform

The application opens on an illustrated storefront with a signboard and sliding glass doors. On the web, hovering slides the doors ajar; a click or tap opens them and the camera moves through the doorway into the location page. On the phone the scene is framed on the door. The tap that opens the doors is also when the app asks for location permission, so the map never has to ask unexpectedly.

The design system is a light, storybook style: Nunito display type, white canvas, one saturated green for headings and primary actions, blue for links, 12 px radius, 2 px borders and no shadows. Every icon is an SVG; no emoji are used anywhere.

The pipeline (`.github/workflows`) runs lint, type check, unit tests, security-rule tests and an end-to-end test against the Firebase emulator on every push; builds and uploads a signed release APK on every push to `main` and attaches it to the release on

every `v*` tag; deploys Firestore rules and indexes; and publishes the documentation site. Vercel deploys production from `main` and a preview for every pull request.

Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** Vitest covers pure logic: price and time formatting, the feed ranking (radius filtering, interest-first ordering, distance), product-code generation, word counting, and the bundle price tables. 19 tests.
- **Security-rule testing:** `@firebase/rules-unit-testing` exercises the Firestore rules on the emulator as a buyer, a seller and a stranger: profile balances cannot go negative, roles are immutable, shops and products are owner-only, a plain swipe creates only a pending like, a super swipe or an auto-matching shop may create the match atomically, only the seller decides a like, only participants read a chat, purchases are self-only, and reviews require a match. 17 tests.
- **Integration (end-to-end) Testing:** One Vitest suite drives the real data layer through the Auth and Firestore emulators: a seller opens a shop and lists two products (spending uploads from the free token), a shopper sees them within 10 km but not from 150 km away, swipes, is refused on a super-only product, the seller accepts, the chat opens with the auto message and a notification, a super swipe claims the super-only product, the auto-matcher makes a plain swipe match instantly, a stock change fans out, a review is written and edited, and a stranger is denied. 8 steps. The same suite was run once against the hosted project to validate the deployed rules and indexes.
- **Performance Testing:** The application records its own evaluation metrics on the device (swipe-to-match, message delivery, availability sync) and shows medians on a hidden metrics page; numbers below come from this instrumentation on a mid-range Android phone on campus Wi-Fi.
- **User Testing:** The three of us and a handful of classmates used the APK and the website against the seeded catalogue of four demo shops and sixty products in Patiala. Their feedback drove the redesign described in the next section.

Metric	Target	Achieved (median)
Swipe $\rightarrow$ match latency	≤ 2 s	≈ 0.4 s
Message delivery (sender to receiver)	≤ 1 s	≈ 0.5 s
Availability sync (seller to buyer alert)	seconds	≈ 1 s
Seller: open a shop	< 5 min	≈ 3 min
CI run (lint, types, unit, rules, e2e)	–	≈ 2.5 min
Signed APK build and publish	–	≈ 3 min

Table 3.1: Performance results summary (mid-semester build, v0.4.6)

Automated check	Count / status
Unit tests	19 passing
Security-rule tests	17 passing
End-to-end steps on the emulator	8 passing
Tagged releases with a signed APK	v0.1.0 to v0.4.6

Table 3.2: Verification summary

3.2 Performance Metrics

3.3 Analysis

- **Objectives met.** All four objectives are met at the prototype stage: the discovery loop, the local real-time marketplace, seller onboarding with camera listings, and the pipeline. Latency targets are met with a wide margin because every write is a single Firestore batch and every read is a live listener.
- **Where we exceeded.** The pipeline turned out to be the most valuable part of the project. Every one of the fourteen releases so far went from commit to a published website and a signed APK without a manual step, which made it cheap to act on user feedback the same day.
- **Challenges.** Firestore has no geographic query, so locality had to be built from geohash range scans and client-side filtering. A server-less design pushed fan-out and quota logic into the client, which in turn pushed the correctness burden onto security rules; `getAfter()` made atomic like-plus-match writes possible, but every rule needed its own test. Profiles created before the swipe economy existed produced a negative balance on the first swipe and were rejected by the rules; the fix was to write absolute balances and backfill missing fields on login. On the phone, GPS and camera were blocked when testing over plain HTTP, which was solved with an HTTPS development server and by asking for permissions at a predictable moment.

- **What changed from the proposal.** The proposal described a Tauri desktop application; the target became a phone app plus a website. Supabase was replaced by Firestore, Firebase Storage by Cloudinary (new Firebase projects need a paid plan for Storage), and a dark “field terminal” design was replaced by the light storybook design after classmates found the dark version dull. Payments and one-time codes are simulated, which is stated in the interface.

Conclusion

4.1 Summary of Work

We set out to make local selling feel like swiping through a shop window. At mid-semester Window is a working product: a shopper picks an area and interests, swipes real products from sellers around Patiala, likes or claims one, and talks to the seller; a seller opens a shop from a phone, photographs products, accepts likes, chats, and pushes stock updates. The same code runs as a website and as a signed Android application, every change passes an automated suite of 44 checks, and every tagged version is published automatically. All four objectives were met.

4.2 Key Findings

- **Finding 1:** A marketplace with no server of its own is viable when the database's rules are treated as the application's contract and tested as rigorously as code; the rule suite caught more real bugs than the unit tests did.
- **Finding 2:** Locality is what makes the swipe feel meaningful. With a 10 km radius and interest-first ranking, testers recognised shops and areas on the cards, which a global feed never produced.
- **Finding 3:** The economics layer (free daily swipes, super swipes that skip the seller's approval, upload tokens for sellers) changed behaviour even though no money moves: testers hoarded super swipes for products they actually wanted, which is exactly the signal a seller needs.

4.3 Future Work and Recommendations

1. Real payments through UPI and a real one-time-code provider, replacing the simulated purchase and 0000 flows, with a small server or cloud functions to hold the secrets.
2. Push notifications when the app is closed, which requires the paid Firebase plan or a third-party push service.
3. Moderation and trust: image screening for the tag policy, seller verification beyond the self-declared flag, and reporting.

4. Recommendation beyond ranking by interest and distance, for example learning from passes and likes [**firestore**].
5. An iOS build from the same Capacitor project, and a Play Store listing with a production signing key and privacy policy.
6. Offline resilience: queueing swipes and messages while the phone has no signal, which Firestore's local cache makes feasible.

4.4 Final Remarks

The most important lesson for us was the value of shipping small, verified increments. Because the pipeline made each release cheap, we could afford to be wrong about the design twice and fix it the same day, and the product that exists now is the one our testers asked for rather than the one we first imagined. Window is a marketplace, but it is also a demonstration that three students with managed services and a disciplined pipeline can build and operate a real two-sided application in half a semester.